

Tugas Besar 2

IF2211 - Strategi Algoritma



Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada Pohon Document Object Model

Kelompok OneStack:

Muhammad Nafis Habibi	13524018
Muhammad Jordan Ferimeison	13524047
Wildan Abdurrahman Ghazali	13524054

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Daftar Isi

Daftar Isi.....	1
BAB 1	
Deskripsi Tugas.....	2
BAB 2	
Landasan Teori.....	3
1. Breadth First Search (BFS).....	3
2. Depth First Search (DFS).....	4
BAB 3	
Aplikasi BFS dan DFS.....	5
1. Penerapan BFS pada DOM Traversal untuk CSS Selector.....	5
2. Penerapan DFS pada DOM Traversal untuk CSS Selector.....	6
BAB 4	
Implementasi dan Pengujian.....	8
1. Implementasi.....	8
a. Implementasi Parsing dan Tokenisasi HTML.....	8
b. Modul Tokenisasi CSS Selector serta Traversal BFS / DFS.....	22
c. Modul Serialisasi DOM Tree.....	34
d. Modul Frontend sebagai Input dan Visualisasi Hasil.....	37
2. Pengujian.....	46
BAB 5	
Kesimpulan dan Saran.....	68
1. Kesimpulan.....	68
2. Saran.....	69
Lampiran.....	70

BAB 1

Deskripsi Tugas

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

Struktur sebuah file HTML pada umumnya meliputi elemen root `<html>` yang berisi dua bagian utama yaitu `<head>` dan `<body>`. Bagian `<head>` berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan `<body>` berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

BAB 2

Landasan Teori

1. Breadth First Search (BFS)

Breadth First Search (BFS) merupakan algoritma pencarian pada struktur data *tree* (pohon) atau graf yang bekerja dengan cara mengunjungi setiap *node* (simpul) dengan memprioritaskan *node* pada level / kedalaman yang sama secara melebar. BFS memeriksa seluruh simpul yang berada pada level yang sama terlebih dahulu sebelum melanjutkan ke level berikutnya.

Implementasi BFS menggunakan struktur data *queue* yang menerapkan prinsip *first in first out* (FIFO). Langkah umum algoritma BFS adalah sebagai berikut:

- Masukkan *root node* ke dalam *queue* (*enqueue*).
- Ambil *node* paling depan dari *queue* (*dequeue*).
- Kunjungi *node* tersebut dan lakukan operasi terhadap *node*.
- enqueue* seluruh *child* dari *node* tersebut.
- Ulangi langkah b sampai *queue* kosong.

Pseudocode dari algoritma BFS adalah sebagai berikut:

```
BFS (root) :  
    buat Queue kosong  
    enqueue (root)  
  
    while Queue tidak kosong:  
        node = dequeue ()  
        kunjungi (node)  
  
        untuk setiap child dari node:  
            enqueue (child)
```

Algoritma BFS pada struktur *tree* dengan n node memiliki kompleksitas waktu $O(n)$. Hasil ini didapatkan dari banyaknya node yang dikunjungi, yaitu n .

2. Depth First Search (DFS)

Depth First Search (DFS) merupakan salah satu algoritma pencarian pada struktur data *tree* (pohon) atau graf yang bekerja dengan cara memprioritaskan menelusuri suatu cabang dan mengunjungi setiap *node* (simpul) sedalam mungkin sebelum kembali mengunjungi cabang lainnya. DFS memprioritaskan kedalaman / level maksimal sebuah cabang hingga mencapai node terdalam, sebelum melakukan *backtracking* untuk mencari cabang lainnya pada *parent node*.

Implementasi DFS dapat menggunakan struktur data *stack* yang menerapkan prinsip *last in first out* (LIFO) maupun *tree* dengan metode rekursi. Langkah algoritma DFS menggunakan *tree* dan rekursi adalah sebagai berikut:

- a. Kunjungi *root node* dan lakukan operasi terhadap node
- b. Untuk setiap child node pada node saat ini, ulangi langkah a

Berikut adalah pseudocode untuk DFS:

```
DFS (node) :  
    kunjungi (node)  
    untuk setiap child dari node:  
        DFS (child)
```

Algoritma DFS pada struktur *tree* dengan n node memiliki kompleksitas waktu $O(n)$. Hasil ini didapatkan dari banyaknya node yang dikunjungi, yaitu n .

BAB 3

Aplikasi BFS dan DFS

1. Penerapan BFS pada DOM Traversal untuk CSS Selector

Breadth First Search diterapkan untuk menelusuri DOM Tree secara melebar berdasarkan level. Traversal dimulai dari root node, kemudian seluruh node pada level yang sama dikunjungi terlebih dahulu sebelum sistem turun ke level berikutnya.

Pada implementasinya, BFS menggunakan struktur data Queue. Mekanisme kerjanya adalah:

- a. Root node dimasukkan ke queue.
- b. Node paling depan queue diambil dan dikunjungi.
- c. Node tersebut dicatat ke traversal log.
- d. Sistem memeriksa apakah node cocok dengan CSS selector.
- e. Jika cocok, node dimasukkan ke daftar hasil.
- f. Seluruh child node dimasukkan ke queue.
- g. Proses berulang sampai queue kosong atau hasil telah mencapai batas top n.

Pseudocode BFS dalam penelusuran CSS Selector adalah sebagai berikut:

```
function DOMNode[] GetNodesBFS(DOMNode root, Func<DOMNode,
bool> selectorMatch):
    Queue queue<DOMNode>;
    DOMNode[] matchedNodes;
    enqueue(root);
    while queue not empty:
        node = dequeue();
        traversalLog.add(node);
        if selectorMatch(node):
            matchedNodes.add(node);
        for each (child in node.children):
            enqueue(node.child);
    return matchedNodes
```

2. Penerapan DFS pada DOM Traversal untuk CSS Selector

Depth First Search diterapkan untuk menelusuri DOM Tree secara mendalam pada satu cabang sebelum berpindah ke cabang lainnya. Metode ini memungkinkan sistem menjangkau elemen-elemen HTML yang berada jauh di dalam nested structure lebih cepat.

Pada implementasinya, DFS menggunakan fungsi rekursif untuk traversal setiap node cabang:

- a. Root node dikunjungi..
- b. Node dicatat ke traversal log.
- c. Sistem memeriksa kecocokan terhadap CSS selector.
- d. Jika cocok, node disimpan sebagai matched node.
- e. Child node dikunjungi
- f. Jika satu cabang habis, algoritma melakukan backtracking.
- g. Proses berulang hingga semua node dikunjungi

Pseudocode DFS dalam penelusuran CSS Selector adalah sebagai berikut:

```
function      DOMNode[]      GetNodesDFS (DOMNode      root,
Func<DOMNode, bool> selectorMatch):
    DOMNode[] matchedNodes;
    TraverseDFS(node, matchedNodes, selectorMatch);
    return result;

procedure      TraverseDFS (DOMNode      node,      DOMNode[]
matchedNodes,      Func<DOMNode, bool> selectorMatch):
    traversalLog.add(node);
    if selectorMatch(node):
        matchedNodes.add(node);
    for each (child in node.children):
        TraverseDFS(child,matchedNodes, selectorMatch)
```


BAB 4

Implementasi dan Pengujian.

1. Implementasi

Aplikasi traversal DOM Tree pada tugas besar ini diimplementasikan menggunakan bahasa pemrograman C# dengan framework ASP.NET Core MVC sebagai backend utama. Pemilihan C# didasarkan pada kemampuannya dalam mendukung pemrograman berorientasi objek, pengelolaan struktur data yang kompleks, serta kemudahan integrasi antara backend logic dan frontend web interface.

Secara umum, implementasi sistem dibagi menjadi empat modul utama, yaitu parsing serta tokenisasi HTML, tokenisasi CSS Selector serta traversal BFS / DFS, serialisasi DOM Tree, dan Modul Frontend sebagai input serta visualisasi hasil.

a. Implementasi Parsing dan Tokenisasi HTML

HTML Parser merupakan komponen yang bertugas mengubah dokumen HTML mentah menjadi struktur data pohon DOM Tree yang dapat diproses oleh algoritma traversal. Pada implementasi ini, parser dibangun secara manual tanpa menggunakan library parser HTML eksternal agar proses pembentukan tree dapat dikontrol sepenuhnya.

Implementasi HTML Parser dibagi menjadi dua tahap utama, yaitu:

i. Tokenisasi HTML

Tokenisasi bertujuan memecah dokumen HTML string menjadi unit-unit kecil berupa token yang lebih mudah diproses. Token yang dikenali antara lain:

- opening tag,
- closing tag,
- self-closing tag,
- attribute,
- text content.

Tokenizer membaca string HTML karakter demi karakter dan memeriksa pola tag.

ii. Parsing token menjadi DOM Tree

Setelah token HTML diperoleh, tahap berikutnya adalah menyusun token tersebut menjadi struktur pohon DOM. Parsing dilakukan menggunakan struktur data Stack untuk menjaga hubungan parent-child antar tag.

Langkah parsing yang diimplementasikan adalah:

- Membuat root node document.
- Jika token berupa opening tag:
 - buat node baru,
 - tambahkan sebagai child current node,
 - push ke stack.
- Jika token berupa closing tag, pop stack.
- Jika token berupa self-closing:
 - buat child node tanpa push.
- Jika token berupa text, buat text node.

Dengan mekanisme ini, setiap tag yang berada di dalam tag lain secara otomatis menjadi child node dari parent yang sesuai.

Berikut adalah kode program HTMLParser.cs :

```
public class HTMLParser
{
    public static async Task<string> Scrape(string url)
    {
        using HttpClient client = new HttpClient();
        client.Timeout = TimeSpan.FromSeconds(15);
        client.DefaultRequestHeaders.UserAgent.ParseAdd(
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 " +
            "(KHTML, like Gecko) Chrome/124.0.0.0
Safari/537.36");
```

```

        string normalizedUrl = NormalizeUrl(url);
        bool inputHasExplicitScheme =
HasExplicitHttpScheme(url);
        try
        {
            return await
client.GetStringAsync(normalizedUrl);
        }
        catch (HttpRequestException ex) when
(!inputHasExplicitScheme &&
normalizedUrl.StartsWith("https://",
StringComparison.OrdinalIgnoreCase) &&
ex.StatusCode is null)
        {
            // Retry dengan http:// jika https gagal
            string httpUrl = "http://" +
normalizedUrl["https://".Length..];
            return await client.GetStringAsync(httpUrl);
        }
    }

    public static DOMTree ParseFromString(string html)
    {
        DOMTree domTree = new DOMTree();
        Tokenizer tokenizer = new Tokenizer();
        var tokens = tokenizer.Tokenize(html);

        Stack<DOMNode> stack = new Stack<DOMNode>();
        stack.Push(domTree.Root);
        foreach (var token in tokens)
        {
            switch (token.Type)
            {
                case TokenType.TagOpen:
                {
                    DOMNode node =
domTree.AddChild(stack.Peek(), token.Value);
                    ApplyAttributes(node, token);
                    if (!token.IsSelfClosing)
                    {
                        stack.Push(node);
                    }
                    break;
                }
            }
        }
    }

```

```

        }
        case TokenType.Text:
        {
            domTree.AddChild(stack.Peek(),
"", token.Value);
            break;
        }
        case TokenType.TagClose:
        {
            CloseNode(stack, token.Value);
            break;
        }
        default:
            break;
    }
}

return domTree;
}

public static async Task<DOMTree>
ParseFromUrl(string url)
{
    string html = await Scrape(url);
    return ParseFromString(html);
}

private static void ApplyAttributes(DOMNode node,
Token token)
{
    node.Attribute = new Dictionary<string,
string>(token.Attributes,
StringComparer.OrdinalIgnoreCase);

    if (token.Attributes.TryGetValue("id", out
string? id))
    {
        node.Id = id;
    }

    if (token.Attributes.TryGetValue("class", out
string? classes))
    {
        node.Class = classes
            .Split(' ',
StringSplitOptions.RemoveEmptyEntries |

```

```

StringSplitOptions.TrimEntries)
        .ToList();
    }
}

private static void CloseNode(Stack<DOMNode> stack,
string closingTag)
{
    bool found = stack.Any(n =>
        string.Equals(n.Tag, closingTag,
StringComparison.OrdinalIgnoreCase));

    if (!found) return;

    while (stack.Count > 1 &&
        !string.Equals(stack.Peek().Tag, closingTag,
StringComparison.OrdinalIgnoreCase))
    {
        stack.Pop();
    }

    if (stack.Count > 1)
        stack.Pop();
}

private static string NormalizeUrl(string url)
{
    if (string.IsNullOrEmpty(url))
        throw new ArgumentException("URL tidak boleh
kosong.", nameof(url));

    string trimmed = url.Trim();

    if (Uri.TryCreate(trimmed, UriKind.Absolute, out
Uri? absoluteUri)
        && (absoluteUri.Scheme == "https" ||
absoluteUri.Scheme == "http"))
    {
        if
(string.IsNullOrEmpty(absoluteUri.Host))
            throw new UriFormatException("Host URL
tidak valid.");

        return absoluteUri.ToString();
    }
}

```

```

        if (Uri.TryCreate(trimmed, UriKind.Absolute, out
absoluteUri))
            throw new UriFormatException("URL harus
menggunakan http:// atau https://.");

        if (trimmed.Contains("://",
StringComparison.Ordinal))
            throw new UriFormatException("Format URL
tidak valid.");

        if (Uri.TryCreate($"https://{trimmed}",
UriKind.Absolute, out absoluteUri) &&
!string.IsNullOrEmpty(absoluteUri.Host))
            return absoluteUri.ToString();

        throw new UriFormatException("Format URL tidak
valid.");
    }

    private static bool HasExplicitHttpScheme(string
url)
    {
        if (!Uri.TryCreate(url.Trim(), UriKind.Absolute,
out Uri? absoluteUri))
            return false;

        return absoluteUri.Scheme == Uri.UriSchemeHttp
|| absoluteUri.Scheme == Uri.UriSchemeHttps;
    }
}

```

Berikut adalah kode program Tokenizer.cs

```

public enum TokenType
{
    TagOpen,
    TagClose,
    Text
}

public class Token
{
    public TokenType Type { get; set; }
    public string Value { get; set; } = string.Empty;
}

```

```

        public Dictionary<string, string> Attributes { get;
set; } = new Dictionary<string,
string>(StringComparer.OrdinalIgnoreCase);
        public bool IsSelfClosing { get; set; }

        public Token()
        {
        }

        public Token(TokenType type, string value)
        {
            Type = type;
            Value = value;
        }
    }

    public class Tokenizer
    {
        private static readonly HashSet<string> VoidTags =
new(StringComparer.OrdinalIgnoreCase)
        {
            "area", "base", "br", "col", "embed", "hr",
"img", "input",
            "link", "meta", "param", "source", "track",
"wbr"
        };

        public List<Token> Tokenize(string html)
        {
            List<Token> tokens = new List<Token>();

            if (string.IsNullOrEmpty(html))
            {
                return tokens;
            }

            int index = 0;
            while (index < html.Length)
            {
                if (html[index] != '<')
                {
                    int textEnd = html.IndexOf('<', index);
                    if (textEnd < 0)
                    {
                        textEnd = html.Length;
                    }
                }
            }
        }
    }

```

```

        AddTextToken(tokens,
html.Substring(index, textEnd - index));
        index = textEnd;
        continue;
    }

    if (StartsWith(html, index, "<!--"))
    {
        int commentEnd = html.IndexOf("-->",
index + 4, StringComparison.Ordinal);
        index = commentEnd >= 0 ? commentEnd + 3
: html.Length;
        continue;
    }

    if (StartsWith(html, index, "<!") )
    {
        int declarationEnd = FindTagEnd(html,
index + 2);
        index = declarationEnd >= 0 ?
declarationEnd + 1 : html.Length;
        continue;
    }

    if (StartsWith(html, index, "<?"))
    {
        int processingEnd = html.IndexOf("?>",
index + 2, StringComparison.Ordinal);
        index = processingEnd >= 0 ?
processingEnd + 2 : html.Length;
        continue;
    }

    int tagEnd = FindTagEnd(html, index + 1);
    if (tagEnd < 0)
    {
        AddTextToken(tokens, html[index..]);
        break;
    }

    string rawTagContent = html.Substring(index
+ 1, tagEnd - index - 1).Trim();
    index = tagEnd + 1;

    if (string.IsNullOrEmpty(rawTagContent))

```

```

        {
            continue;
        }

        if (rawTagContent[0] == '/')
        {
            string closingTagName =
ExtractTagName(rawTagContent[1..]);
            if
(!string.IsNullOrEmpty(closingTagName))
            {
                tokens.Add(new
Token(TokenType.TagClose, closingTagName));
            }

            continue;
        }

        bool isSelfClosing =
rawTagContent.EndsWith("/", StringComparison.Ordinal);
        if (isSelfClosing)
        {
            rawTagContent =
rawTagContent[..^1].TrimEnd();
        }

        string tagName =
ExtractTagName(rawTagContent);
        if (string.IsNullOrEmpty(tagName))
        {
            continue;
        }

        Dictionary<string, string> attributes =
ParseAttributes(rawTagContent[tagName.Length..]);
        Token openToken = new()
        {
            Type = TokenType.TagOpen,
            Value = tagName,
            Attributes = attributes,
            IsSelfClosing = isSelfClosing ||
VoidTags.Contains(tagName)
        };
        tokens.Add(openToken);

        if (!openToken.IsSelfClosing &&

```

```

IsRawTextElement(tagName))
    {
        int closingTagStart =
FindClosingTagStart(html, index, tagName);
        if (closingTagStart < 0)
        {
            AddTextToken(tokens, html[index..],
preserveWhitespace: true);
            tokens.Add(new
Token(TokenType.TagClose, tagName));
            break;
        }

        AddTextToken(tokens,
html.Substring(index, closingTagStart - index),
preserveWhitespace: true);

        int closingTagEnd = FindTagEnd(html,
closingTagStart + 2);
        if (closingTagEnd < 0)
        {
            tokens.Add(new
Token(TokenType.TagClose, tagName));
            break;
        }

        string closingTagName =
ExtractTagName(html.Substring(closingTagStart + 2,
closingTagEnd - closingTagStart - 2));
        if
(!string.IsNullOrEmpty(closingTagName))
        {
            tokens.Add(new
Token(TokenType.TagClose, closingTagName));
        }

        index = closingTagEnd + 1;
    }

    return tokens;
}

private static void AddTextToken(List<Token> tokens,
string rawText, bool preserveWhitespace = false)
{

```

```

        if (string.IsNullOrEmpty(rawText)) return;

        if (!preserveWhitespace &&
string.IsNullOrEmptyWhiteSpace(rawText)) return;

        string decodedText = preserveWhitespace ?
rawText : WebUtility.HtmlDecode(rawText);
        if (string.IsNullOrEmpty(decodedText)) return;

        tokens.Add(new Token(TokenType.Text,
decodedText));
    }
    private static int FindTagEnd(string html, int
startIndex)
    {
        bool inQuotes = false;
        char quoteCharacter = '\0';

        for (int index = startIndex; index <
html.Length; index++)
        {
            char current = html[index];

            if ((current == '"' || current == '\') &&
(!inQuotes || current == quoteCharacter))
            {
                if (inQuotes && current ==
quoteCharacter)
                {
                    inQuotes = false;
                    quoteCharacter = '\0';
                }
                else if (!inQuotes)
                {
                    inQuotes = true;
                    quoteCharacter = current;
                }

                continue;
            }

            if (!inQuotes && current == '>')
            {
                return index;
            }
        }
    }

```

```

        return -1;
    }

    private static string ExtractTagName(string
tagContent)
    {
        string trimmed = tagContent.Trim();
        if (string.IsNullOrEmpty(trimmed))
        {
            return string.Empty;
        }

        int endIndex = 0;
        while (endIndex < trimmed.Length &&
!char.IsWhiteSpace(trimmed[endIndex]) &&
trimmed[endIndex] != '/')
        {
            endIndex++;
        }

        return trimmed[..endIndex].ToLowerInvariant();
    }

    private static Dictionary<string, string>
ParseAttributes(string rawAttributes)
    {
        Dictionary<string, string> attributes =
new(StringComparer.OrdinalIgnoreCase);
        int index = 0;

        while (index < rawAttributes.Length)
        {
            while (index < rawAttributes.Length &&
char.IsWhiteSpace(rawAttributes[index]))
            {
                index++;
            }

            if (index >= rawAttributes.Length ||
rawAttributes[index] == '/')
            {
                break;
            }

            int nameStart = index;

```

```

        while (index < rawAttributes.Length &&
!char.IsWhiteSpace(rawAttributes[index]) &&
            rawAttributes[index] != '=' &&
            rawAttributes[index] != '/')
        {
            index++;
        }

        string attributeName =
rawAttributes[nameStart..index];
        if
(string.IsNullOrEmpty(attributeName))
        {
            break;
        }

        while (index < rawAttributes.Length &&
char.IsWhiteSpace(rawAttributes[index]))
        {
            index++;
        }

        string attributeValue = "true";
        if (index < rawAttributes.Length &&
rawAttributes[index] == '=')
        {
            index++;

            while (index < rawAttributes.Length &&
char.IsWhiteSpace(rawAttributes[index]))
            {
                index++;
            }

            if (index < rawAttributes.Length &&
(rawAttributes[index] == '"' || rawAttributes[index] ==
'\'))
            {
                char quote = rawAttributes[index];
                index++;
                int valueStart = index;

                while (index < rawAttributes.Length
&& rawAttributes[index] != quote)
                {

```

```

        index++;
    }

    attributeValue =
rawAttributes[valueStart..Math.Min(index,
rawAttributes.Length)];
    if (index < rawAttributes.Length &&
rawAttributes[index] == quote)
    {
        index++;
    }
    else
    {
        int valueStart = index;
        while (index < rawAttributes.Length
&&
!char.IsWhiteSpace(rawAttributes[index]))
        {
            index++;
        }

        attributeValue =
rawAttributes[valueStart..index];
    }

    attributes[attributeName.ToLowerInvariant()]
= WebUtility.HtmlDecode(attributeValue);
}

return attributes;
}

private static bool StartsWith(string source, int
startIndex, string value)
{
    return
source.AsSpan(startIndex).StartsWith(value,
StringComparison.Ordinal);
}

private static bool IsRawTextElement(string tagName)
{
    return tagName.Equals("script",

```

```

StringComparison.OrdinalIgnoreCase) ||
    tagName.Equals("style",
StringComparison.OrdinalIgnoreCase);
    }

    private static int FindClosingTagStart(string html,
int startIndex, string tagName)
    {
        string closingTag = $"</{tagName}";
        int searchIndex = startIndex;

        while (searchIndex < html.Length)
        {
            int candidate = html.IndexOf(closingTag,
searchIndex, StringComparison.OrdinalIgnoreCase);
            if (candidate < 0)
            {
                return -1;
            }

            int boundaryIndex = candidate +
closingTag.Length;
            if (boundaryIndex >= html.Length ||
char.IsWhiteSpace(html[boundaryIndex]) ||
html[boundaryIndex] == '>')
            {
                return candidate;
            }

            searchIndex = candidate + 1;
        }

        return -1;
    }
}

```

b. Modul Tokenisasi CSS Selector serta Traversal BFS / DFS

CSS Selector yang didapatkan dari pengguna merupakan sebuah string yang harus ditokenisasi guna mendapatkan selector yang valid. Token yang diidentifikasi meliputi:

- i. tag selector,
- ii. id selector,
- iii. class selector,
- iv. universal selector,
- v. attribute selector ([attribute="value"]),
- vi. combinator (>, spasi, +, ~).

Setelah token dari selector didapatkan, program melakukan pencocokan terhadap node DOM yang sedang dikunjungi pada algoritma BFS dan DFS yang telah dijelaskan pada bab sebelumnya dengan menjadikan token sebagai filter match selector.

Berikut adalah kode program CSSSelector.cs :

```
#nullable enable
namespace DOMTreeTraversal.Models;

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;

public enum TraversalMethod
{
    DFS,
    BFS
}

public static class CSSSelector
{
    public static List<DOMNode> traversalNodes = new
    List<DOMNode>();
    public static (List<DOMNode>, TimeSpan)
    QuerySelector(DOMTree tree, string selector, int
    maxSelected, TraversalMethod method =
    TraversalMethod.DFS)
    {
        return QuerySelector(tree.Root, selector,
```

```

maxSelected, method);
    }
    public static (List<DOMNode>, TimeSpan)
QuerySelector(DOMNode root, string selector, int
maxSelected, TraversalMethod method =
TraversalMethod.DFS)
    {
        traversalNodes = new();
        if (string.IsNullOrEmpty(selector)) return
(new List<DOMNode>(), new TimeSpan());

        var tokens = TokenizeSelector(selector);
        if (tokens.Count == 0) return (new
List<DOMNode>(), new TimeSpan());

        DateTime startTime = DateTime.Now;

        var currNodes = GetNodes(root, method, n =>
MatchSelector(n, tokens[0]));
        if (tokens.Count == 1 && currNodes.Count >
maxSelected)
        {
            var temp = new List<DOMNode>();
            int i = 0;
            foreach (var nodes in currNodes)
            {
                if (i < maxSelected) temp.Add(nodes);
                else break;
                i++;
            }
            currNodes = temp;
        }

        // Filterrrrrrrrrrr
        int lastToken = tokens.Count - 2;

        for (int i = 1; i < tokens.Count; i += 2)
        {
            // ganjil jangan!
            if (i + 1 >= tokens.Count) break;

            bool isLastToken = i == lastToken;
            string combinator = tokens[i];
            string nextSelector = tokens[i + 1];
            HashSet<DOMNode> nextNodes = new
HashSet<DOMNode>();

```

```

        foreach (var node in currNodes)
        {
            if (maxSelected == 0) break;
            traversalNodes.Add(node);

            if (combinator == " ") // Descendant
            {
                var descendants = GetNodes(node,
method, n => MatchSelector(n, nextSelector));
                foreach (var desc in descendants)
                {
                    if (maxSelected == 0) break;
                    traversalNodes.Add(desc);
                    if (MatchSelector(desc,
nextSelector))
                    {
                        if (nextNodes.Add(desc) &&
isLastToken) maxSelected--;
                    }
                }
            }
            else if (combinator == ">") // Child
            {
                foreach (var child in node.Children)
                {
                    if (maxSelected == 0) break;
                    traversalNodes.Add(child);
                    if (MatchSelector(child,
nextSelector))
                    {
                        if (nextNodes.Add(child) &&
isLastToken) maxSelected--;
                    }
                }
            }
            else if (combinator == "+") // Adjacent
Sibling
            {
                var sibling =
GetNextElementSibling(node);
                if (sibling != null)
traversalNodes.Add(sibling);
                if (sibling != null &&
MatchSelector(sibling, nextSelector))
                {

```

```

        if (nextNodes.Add(sibling) &&
isLastToken) maxSelected--;
    }
}
else if (combinator == "~") // General
Sibling
{
    var siblings =
GetNextElementSiblings(node);
    foreach (var sibling in siblings)
    {
        if (maxSelected == 0) break;
        traversalNodes.Add(sibling);
        if (MatchSelector(sibling,
nextSelector))
        {
            if (nextNodes.Add(sibling)
&& isLastToken) maxSelected--;
        }
    }
}

currNodes = nextNodes.ToList();
}

DateTime endTime = DateTime.Now;
TimeSpan runTime = endTime - startTime;

return (currNodes, runTime);
}

private static List<DOMNode> GetNodes(DOMNode
startNode, TraversalMethod method, Func<DOMNode, bool>
condition)
{
    return method == TraversalMethod.DFS
        ? GetNodesDFS(startNode, condition)
        : GetNodesBFS(startNode, condition);
}

private static List<DOMNode> GetNodesDFS(DOMNode
node, Func<DOMNode, bool> condition)
{
    var result = new List<DOMNode>();
    TraverseDFS(node, result, condition);
}

```

```

        return result;
    }

    private static void TraverseDFS(DOMNode node,
    List<DOMNode> result, Func<DOMNode, bool> condition)
    {
        traversalNodes.Add(node);
        if (condition(node)) result.Add(node);
        foreach (var child in node.Children)
        TraverseDFS(child, result, condition);
    }

    private static List<DOMNode> GetNodesBFS(DOMNode
    startNode, Func<DOMNode, bool> condition)
    {
        var result = new List<DOMNode>();
        var queue = new Queue<DOMNode>();
        queue.Enqueue(startNode);

        while (queue.Count > 0)
        {
            var current = queue.Dequeue();
            traversalNodes.Add(current);
            if (condition(current)) result.Add(current);

            foreach (var child in current.Children)
            queue.Enqueue(child);
        }

        return result;
    }

    private static bool MatchSelector(DOMNode node,
    string selector)
    {
        if (node.IsTextNode()) return false;
        if (selector == "*") return true;

        string? tag = null;
        string? id = null;
        List<string> classes = new List<string>();
        List<(string attr, string op, string val)>
        attributes = new List<(string, string, string)>();

        int i = 0;
        while (i < selector.Length)

```

```

        {
            if (selector[i] == '#') // ID
            {
                int start = ++i;
                while (i < selector.Length &&
selector[i] != '#' && selector[i] != '.' && selector[i]
!= '[') i++;
                id = selector.Substring(start, i -
start);
            }
            else if (selector[i] == '.') // Class
            {
                int start = ++i;
                while (i < selector.Length &&
selector[i] != '#' && selector[i] != '.' && selector[i]
!= '[') i++;
                classes.Add(selector.Substring(start, i
- start));
            }
            else if (selector[i] == '[') // Attribute
            {
                int start = ++i;
                while (i < selector.Length &&
selector[i] != ']') i++; // Asumsi gaada closing bracket
error
                string attrContent =
selector.Substring(start, i - start);

attributes.Add(ParseAttribute(attrContent));

                if (i < selector.Length && selector[i]
== ']') i++;
            }
            else // Tag
            {
                int start = i;
                while (i < selector.Length &&
selector[i] != '#' && selector[i] != '.' && selector[i]
!= '[') i++;
                tag = selector.Substring(start, i -
start);
            }
        }

// Bandingin tag selector dan tag node

```

```

        if (tag != null && tag != "*" &&
!string.Equals(node.Tag, tag,
StringComparison.OrdinalIgnoreCase)) return false;

        // Bandingin id selector dan id node
        if (id != null && !string.Equals(node.Id, id,
StringComparison.OrdinalIgnoreCase)) return false;

        // Bandingin class node dan class selector
        foreach (var c in classes)
        {
            if (!node.Class.Contains(c)) return false;
        }

        // Bandingin attribute node dan attribute
selector
        foreach (var a in attributes)
        {
            if (!node.Attribute.TryGetValue(a.attr, out
var val))
                return false;

            if (a.op == "=" && !string.Equals(val,
a.val, StringComparison.OrdinalIgnoreCase))
                return false;

            if (a.op == "~=")
            {
                var words = val.Split(new[] { ' ', '\t',
'\r', '\n' }, StringSplitOptions.RemoveEmptyEntries);
                if(!words.Contains(a.val,
StringComparer.OrdinalIgnoreCase))
                    return false;
            }

            if (a.op == "|=")
            {
                if(!string.Equals(val, a.val,
StringComparison.OrdinalIgnoreCase) &&
!val.StartsWith(a.val + "-",
StringComparison.OrdinalIgnoreCase))
                    return false;
            }

            if (a.op == "^=" && !val.StartsWith(a.val,
StringComparison.OrdinalIgnoreCase))

```

```

        return false;

        if (a.op == "$=" && !val.EndsWith(a.val,
StringComparison.OrdinalIgnoreCase))
            return false;

        if (a.op == "*=" && !val.Contains(a.val,
StringComparison.OrdinalIgnoreCase))
            return false;
    }

    return true;
}

private static (string, string, string)
ParseAttribute(string contents)
{
    (string, string, string) parsed = new ("", "",
    "");

    int opIdx = contents.IndexOf("=");
    string op = "=";

    if (opIdx == -1) parsed = (contents.Trim(), "",
    "");
    else
    {
        string modifier = "~|^$*";
        int attrEnd = opIdx - 1;
        int valStart = opIdx + 1;

        if (opIdx > 0 &&
modifier.Contains(contents[attrEnd]))
        {
            op = contents.Substring(attrEnd, 2);
            attrEnd--;
        }

        string attr = contents.Substring(0, attrEnd
+ 1).Trim();
        string val =
contents.Substring(valStart).Trim();

        if (val.Length >= 2 &&
((val.StartsWith("\"") && val.EndsWith("\"")) ||
(val.StartsWith("'") && val.EndsWith("'"))))
            val = val.Substring(1, val.Length - 2);
    }
}

```

```

        parsed = (attr, op, val);
    }

    return parsed;
}

private static List<string> TokenizeSelector(string
selector)
{
    List<string> tokens = new List<string>();
    bool insideBrackets = false;
    bool insideQuotes = false;
    char quoteChar = '\\0';

    string currentToken = "";

    for (int i = 0; i < selector.Length; i++)
    {
        char c = selector[i];

        if (insideQuotes)
        {
            currentToken += c;
            if (c == quoteChar) insideQuotes =
false;
        }
        else if (insideBrackets)
        {
            currentToken += c;
            if (c == '"' || c == '\\')
            {
                insideQuotes = true;
                quoteChar = c;
            }
            else if (c == ']') insideBrackets =
false;
        }
        else
        {
            if (c == '"' || c == '\\')
            {
                insideQuotes = true;
                quoteChar = c;
                currentToken += c;
            }

```

```

        else if (c == '[')
        {
            insideBrackets = true;
            currentToken += c;
        }
        else if (c == ' ' || c == '\t' || c ==
'\r' || c == '\n')
        {
            if
(!string.IsNullOrEmpty(currentToken))
            {
                tokens.Add(currentToken);
                currentToken = "";
            }
            if (tokens.Count > 0 &&
tokens.Last() != ">" && tokens.Last() != "+"
&& tokens.Last() != "~" &&
tokens.Last() != " ")
            {
                tokens.Add(" ");
            }
        }
        else if (c == '>' || c == '+' || c ==
'~')
        {
            if
(!string.IsNullOrEmpty(currentToken))
            {
                tokens.Add(currentToken);
                currentToken = "";
            }

            if (tokens.Count > 0 &&
tokens.Last() == " ")
                tokens.RemoveAt(tokens.Count -
1);

            tokens.Add(c.ToString());
        }
        else currentToken += c;
    }

    if (!string.IsNullOrEmpty(currentToken))
tokens.Add(currentToken);

```

```

        while (tokens.Count > 0 && tokens.Last() == " ")
tokens.RemoveAt(tokens.Count - 1);

        return tokens;
    }

    private static DOMNode?
GetNextElementSibling(DOMNode node)
    {
        if (node.Parent == null) return null;
        var siblings = node.Parent.Children;
        int index = siblings.IndexOf(node);

        if (index >= 0)
        {
            for (int i = index + 1; i < siblings.Count;
i++)
            {
                if (!siblings[i].IsTextNode()) return
siblings[i];
            }
        }
        return null;
    }

    private static List<DOMNode>
GetNextElementSiblings(DOMNode node)
    {
        var result = new List<DOMNode>();
        if (node.Parent == null) return result;

        var siblings = node.Parent.Children;
        int index = siblings.IndexOf(node);

        if (index >= 0)
        {
            for (int i = index + 1; i < siblings.Count;
i++)
            {
                traversalNodes.Add(siblings[i]);
                if (!siblings[i].IsTextNode())
result.Add(siblings[i]);
            }
        }
        return result;
    }

```

```
}
```

c. Modul Serialisasi DOM Tree

Modul ini memetakan setiap node pada DOMTree ke dalam bentuk node serializable yang hanya menyimpan informasi-informasi penting, yaitu:

- i. identitas node,
- ii. nama tag HTML,
- iii. text content,
- iv. atribut id,
- v. atribut class,
- vi. atribut tambahan,
- vii. indeks parent,
- viii. daftar indeks child.

Dengan pendekatan tersebut, hubungan antar node tidak lagi disimpan dalam bentuk referensi objek, melainkan dalam bentuk relasi indeks numerik. Setiap node diberikan `NodeId` unik sehingga keseluruhan pohon dapat disimpan sebagai sekumpulan pasangan data `NodeId` $\rightarrow$ `NodeInfo`.

Proses pemetaan seluruh node dilakukan menggunakan traversal Breadth First Search agar semua node dapat dikunjungi satu kali secara sistematis. Selama proses ini, sistem juga menghitung kedalaman masing-masing node terhadap root untuk memperoleh informasi kedalaman maksimum DOM Tree. Nilai kedalaman maksimum ini kemudian disimpan sebagai salah satu metadata hasil serialisasi.

Hasil akhir dari modul ini adalah sebuah representasi pohon DOM dalam format JSON yang memiliki tiga komponen utama, yaitu:

- `RootId`, sebagai penanda node akar,
- `MaxLevel`, sebagai kedalaman maksimum pohon,
- `Nodes`, yaitu kumpulan seluruh node yang telah dipetakan.

Berikut adalah kode program MappedDOMTree.cs

```
namespace DOMTreeTraversal.Models;

public class MappedDomTree
{
    public int RootId { get; set; }
    public int MaxLevel { get; set; }
    public Dictionary<int, MappedDomNode> Nodes { get;
set; }
    public MappedDomTree()
    {
        RootId = 0;
        MaxLevel = 0;
        Nodes = [];
    }
    public MappedDomTree(DOMTree tree)
    {
        RootId = tree.Root.NodeId;
        MaxLevel = 0;
        Nodes = [];
        Queue<DOMNode> bfs = new();
        Dictionary<DOMNode, int> level = [];
        bfs.Enqueue(tree.Root);
        while (bfs.Count > 0)
        {
            DOMNode node = bfs.Dequeue();
            MappedDomNode jsonable = new()
            {
                Index = node.NodeId,
                Tag = node.Tag,
                Text = node.Text,
                Id = node.Id,
                Class = [... node.Class],
                Attribute = node.Attribute,
                Children = []
            };
            if (node.Text.Length > 0)
            {
                // Console.WriteLine(node.Text);
            }
            if (node.Parent != null)
            {
                jsonable.Parent = node.Parent.NodeId;
            }
            Nodes[jsonable.Parent].Children.Add(node.NodeId);
        }
    }
}
```

```

        level[node] = level[node.Parent] + 1;
        MaxLevel = Math.Max(MaxLevel,
level[node]);
    }
    else
    {
        jsonable.Parent = -1;
        level[node] = 0;
        MaxLevel = Math.Max(MaxLevel,
level[node]);
    }

    Nodes[jsonable.Index] = jsonable;
    foreach (var child in node.Children)
    {
        bfs.Enqueue(child);
    }
}

}

}

public class MappedDomNode
{
    public int Index { get; set; }
    public string Tag { get; set; }
    public string Text { get; set; }
    public string Id { get; set; }
    public List<string> Class { get; set; }
    public Dictionary<string, string> Attribute { get;
set; }
    public List<int> Children { get; set; }
    public int Parent { get; set; }

    public MappedDomNode()
    {
        Index = -1;
        Tag = "";
        Text = "";
        Id = "";
        Class = [];
        Attribute = [];
        Children = [];
    }
}

```

d. Modul Frontend sebagai Input dan Visualisasi Hasil

Frontend dibangun berbasis halaman web sehingga seluruh proses input dan output dapat dilakukan secara interaktif dalam browser. Implementasi antarmuka dirancang ke dalam beberapa bagian utama.

Bagian pertama adalah panel input HTML. Sistem mendukung tiga jenis input, yaitu URL website, file HTML lokal, dan teks HTML manual. Bagian kedua adalah panel pengaturan traversal, yang berisi:

- pilihan algoritma Breadth First Search atau Depth First Search,
- input CSS selector,
- pilihan jumlah hasil pencarian yang ingin ditampilkan.

Bagian ketiga adalah panel visualisasi DOM Tree. Objek DOM Tree dikirim dalam bentuk string JSON. Objek diserialisasi dengan library JSON dari Newtonsoft. Setelah backend mengirimkan hasil serialisasi JSON, frontend membaca struktur data tersebut dan membangun representasi pohon DOM secara rekursif menggunakan JavaScript dengan cara membuat elemen HTML yang di-style dengan CSS. Karena setiap node pada JSON telah memiliki identitas, parent, dan child, frontend dapat menyusun kembali hubungan antar node dan menampilkannya dalam bentuk pohon.

Frontend memberikan penandaan visual yang berbeda pada node-node tertentu. Node yang hanya dilalui traversal akan ditandai sebagai jalur penelusuran algoritma, sedangkan node yang memenuhi selector akan diberi highlight khusus sebagai hasil pencarian. Animasi BFS/DFS akan berjalan secara otomatis setelah melakukan query CSS selector. Animasi dapat dimulai ulang atau di-skip jika diinginkan.

Frontend juga menampilkan statistik traversal yang diperoleh dari backend, antara lain:

- waktu pencarian,
- jumlah node yang dikunjungi,
- kedalaman maksimum tree,
- jumlah node yang berhasil ditemukan.

Berikut adalah kode program HomeController.cs:

```

using System.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using DOMTreeTraversal.Models.View;
using DOMTreeTraversal.Models.Input;
using DOMTreeTraversal.Test;
using DOMTreeTraversal.Models;
using Newtonsoft.Json;

namespace DOMTreeTraversal.Controllers;

public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();
    }

    public IActionResult About()
    {
        return View();
    }

    [HttpPost]
    public async Task<IActionResult>
DomTreeResult([Bind("HtmlLink", "HtmlText",
"InputType")] HtmlInputModel model, IFormFile htmlFile)
    {
        DOMTree domTree = new();
        try
        {
            switch (model.InputType)
            {
                case "link":
                    domTree = await
HTMLParser.ParseFromUrl(model.HtmlLink ?? "");
                    break;
                case "file":
                    if (htmlFile != null &&
htmlFile.Length > 0)
                    {
                        using var reader = new
StreamReader(htmlFile.OpenReadStream());
                        string htmlContent = await
reader.ReadToEndAsync();
                        domTree =
HTMLParser.ParseFromString(htmlContent);

```

```

        }
        break;
        case "text":
            domTree =
HTMLParser.ParseFromString(model.HtmlText ?? "");
            break;
        }
    }
    catch (Exception e)
    {
        domTree = new();
        Console.WriteLine(e.ToString());
    }

    return View(new DomTreeViewModel(domTree));
}

[HttpPost]
public IActionResult
CssSelectorResult([Bind("DomTreeJson", "CssSelector",
"TraversalType", "ResultCount")] CssSelectorInputModel
model)
{
    MappedDomTree domTree;

    if (model.DomTreeJson != null)
    {
        domTree =
JsonConvert.DeserializeObject<MappedDomTree>(model.DomT
reeJson) ?? new MappedDomTree();
    }
    else
    {
        domTree = new MappedDomTree();
    }
    Console.WriteLine(model.ResultCount);
    var (selectedNode, timeSpan) =
CSSSelector.QuerySelector(
        new DOMTree(domTree),
        model.CssSelector ?? "",
        model.ResultCount ?? 1,
        model.TraversalType == "bfs" ?
TraversalMethod.BFS : TraversalMethod.DFS
    );
    var traversedNode = CSSSelector.traversalNodes;

```

```

        List<int> selectedNodeIndex = new ();
        List<int> traversedNodeIndex = new ();
        foreach (var node in selectedNode)
        {
            if (node.Parent == null) continue;
            selectedNodeIndex.Add(node.NodeId);
        }
        foreach (var node in traversedNode)
        {
            if (node.Parent == null) continue;
            traversedNodeIndex.Add(node.NodeId);
        }
        return View(new CssSelectorViewModel
        {
            DomTreeJson = model.DomTreeJson,
            CssSelector = model.CssSelector,
            TraversalType = model.TraversalType,
            ResultCount = model.ResultCount,
            TimeSpanSecond = timeSpan.TotalSeconds,
            SelectedJson =
                JsonConvert.SerializeObject(selectedNodeIndex),
            TraversalJson =
                JsonConvert.SerializeObject(traversedNodeIndex)
        });
    }

    [ResponseCache(Duration = 0, Location =
ResponseCacheLocation.None, NoStore = true)]
    public IActionResult Error()
    {
        return View(new ErrorViewModel { RequestId =
Activity.Current?.Id ?? HttpContext.TraceIdentifier });
    }
}

```

Berikut adalah kode program site.js

```

// Please see documentation at
https://learn.microsoft.com/aspnet/core/client-side/bundling-and-minification
// for details on configuring this project to bundle
and minify static web assets.

```

```

// Write your JavaScript code.
function build(element =
document.createElement("div"), htmlTree, infoMap =
new Map()) {
    let mainContainer =
document.createElement("div");
    mainContainer.classList.add("w-100", "d-flex");
    mainContainer.style = "min-width: fit-content;";
    element.appendChild(mainContainer);
    let currentNode =
htmlTree.Nodes[htmlTree.RootId];
    let nodeStack = [currentNode];
    let childContainerMap = new Map();
    while (nodeStack.length > 0) {
        // Get node from stack
        currentNode = nodeStack.pop()

        // Make new element
        let newElement =
document.createElement("div");
        newElement.classList.add("flex-grow-1");

        // Append to parent
        if (currentNode.Parent == -1) {
            mainContainer.appendChild(newElement);
        }
        else {

childContainerMap[currentNode.Parent].appendChild(new
Element);
            if
(htmlTree.Nodes[currentNode.Parent].Parent == -1) {
                // do nothing
            }
            else if
(htmlTree.Nodes[currentNode.Parent].Children.length
== 1) {
                let topEdge =
document.createElement("div");

topEdge.classList.add("vertical-edge");
                newElement.appendChild(topEdge);
            }
            else {
                let edgeDiv =

```

```

document.createElement("div");
    edgeDiv.classList.add("d-grid");
    newElement.appendChild(edgeDiv);
    let i =
htmlTree.Nodes[currentNode.Parent].Children.findIndex
((j) => j == currentNode.Index);
    if (i > 0 && i <
htmlTree.Nodes[currentNode.Parent].Children.length -
1) {
        let topLeftEdge =
document.createElement("div");
        let topRightEdge =
document.createElement("div");

topLeftEdge.classList.add("top-to-left-edge");

topRightEdge.classList.add("top-to-right-edge");
        topLeftEdge.style = "grid-column:
1;"
        topRightEdge.style =
"grid-column: 2;"

        edgeDiv.appendChild(topLeftEdge);

edgeDiv.appendChild(topRightEdge);
    }
    else if (i == 0) {
        let topLeftEdge =
document.createElement("div");
        let topRightEdge =
document.createElement("div");

topRightEdge.classList.add("only-top-to-right-edge");
        topLeftEdge.style = "grid-column:
1;"
        topRightEdge.style =
"grid-column: 2;"

        edgeDiv.appendChild(topLeftEdge);

edgeDiv.appendChild(topRightEdge);
    }
    else {
        let topLeftEdge =
document.createElement("div");
        let topRightEdge =

```

```

document.createElement("div");

topLeftEdge.classList.add("only-top-to-left-edge");
topLeftEdge.style = "grid-column:
1;"
topRightEdge.style =
"grid-column: 2;"
edgeDiv.appendChild(topLeftEdge);

edgeDiv.appendChild(topRightEdge);
    }
}

// Create info container
let infoContainer =
document.createElement("div");
infoContainer.classList.add("w-100",
"px-2");
newElement.appendChild(infoContainer);

let infos =
document.createElement("div");
infos.classList.add("node");
infoContainer.appendChild(infos);
infoMap[currentNode.Index] = infos;

// Create infos
let tagText =
document.createElement("p");
if (currentNode.Tag != "") {
    tagText.textContent = "<" +
currentNode.Tag + ">";
}
tagText.classList.add("m-auto", "h5")
infos.appendChild(tagText);
let separator =
document.createElement("hr");
separator.classList.add("my-1")
infos.appendChild(separator);

if (currentNode.Id != "") {
    let idText =
document.createElement("p");
    idText.textContent = "id: \"\" +
currentNode.Id + "\"";
    idText.classList.add("text-start",

```

```

"my-0");
        infos.appendChild(idText);
    }

    if (currentNode.Class.length > 0) {
        let classText =
document.createElement("p");
        classText.textContent = "class: \"" +
currentNode.Class + "\"";
        classText.classList.add("text-start",
"my-0");
        infos.appendChild(classText);
    }

Object.keys(currentNode.Attribute).forEach(function(k
ey,index) {
    if (key == "id" || key == "class")
return;
    let attributeText =
document.createElement("p");
    attributeText.textContent = key + ":
\"" + currentNode.Attribute[key] + "\"";

attributeText.classList.add("text-start", "my-0");
    infos.appendChild(attributeText);
});

    if (currentNode.Text != "") {
        let textText =
document.createElement("p");
        let innerText =
currentNode.Text.replace(/,/g, ", ");
        console.log(innerText);
        textText.textContent = "innerText:
\"" + innerText + "\"";
        textText.classList.add("text-start",
"my-0");
        infos.appendChild(textText);
    }

    if (currentNode.Children.length > 0) {
        let bottomEdge =
document.createElement("div");

bottomEdge.classList.add("vertical-edge");

```

```

        newElement.appendChild(bottomEdge);
    }
}

// Create child container
let childContainer =
document.createElement("div");
newElement.appendChild(childContainer);
childContainer.classList.add("w-100",
"d-flex");
childContainer.style = "min-width:
fit-content;";
childContainerMap[currentNode.Index] =
childContainer;

// Next Elements
for (let i = currentNode.Children.length - 1;
i >= 0; i--) {
    let index = currentNode.Children[i];
    let nextNode = htmlTree.Nodes[index];
    nodeStack.push(nextNode);
}
}

function SanitizeJsonString(rawString) {
    return rawString
        .replace(/[\x00-\x1F\x7F-\x9F]/g, (chr) => {
            // Map common control characters to their
            escaped versions
            const charMap = {
                '\b': '\\b',
                '\f': '\\f',
                '\n': '\\n',
                '\r': '\\r',
                '\t': '\\t',
            };
            return charMap[chr] || ' ';
        });
};

```

2. Pengujian

Berikut adalah beberapa kasus uji yang akan digunakan :

No.	Tipe	Input
1	Text HTML	<pre><!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>My Simple Website</title> <style> body { font-family: Arial, sans-serif; margin: 20px; background-color: #f9f9f9; } header { background-color: #4CAF50; color: white; padding: 10px; text-align: center; } nav a { margin: 0 10px; text-decoration: none; color: white; } section { margin-top: 20px; } footer { margin-top: 20px; text-align: center; font-size: 0.9em; color: #555; } </style> </head> <body> <header> <h1>Welcome to My Website</h1></pre>

		<pre> <nav> Home About Contact </nav> </header> <section id="home"> <h2>Home</h2> <p>This is a simple HTML website example. You can add text, images, and links here.</p> </section> <section id="about"> <h2>About</h2> <p>My name is Alex, and I'm learning web development. This is my first HTML project.</p> </section> <section id="contact"> <h2>Contact</h2> <p>Email me at example@ema il.com</p> </section> <footer> &copy; 2026 My Simple Website </footer> </body> </html> </pre>
	DOM Tree:	

	DOMTreeTraversal Home About DOM Tree Result Insert CSS Selector Traversal Type [BFS ▼] Result Count 5 Search Kedalaman Maksimal: 6 <html> lang="en" <head> <meta> name: viewport; content: width=device-width, initial-scale=1.0; <title> innerHTML: "My Simple Website" <style> innerHTML: "body { font-family: Arial, sans-serif; margin: 20px; background-color: #f9f9f9; } header { background-color: #4caf50; color: white; padding: 10px; text-align: center; margin: 10px 0 10px 0; text-decoration: none; }"; <h1> innerHTML: "Welcome to My Website" <nav> <a> href="home" innerHTML: "Home" <a> href="about" innerHTML: "About" <a> href="contact" innerHTML: "Contact" <h2> innerHTML: "Home" <p> innerHTML: "This is a simple HTML website example. You can add text, images, and links here." src: "https://via.placeholder.com/400x200"; alt: "Sample image" <h2> innerHTML: "About" <p> innerHTML: "My name is Alex and I'm learning web development. This is my first HTML project." <h2> innerHTML: "Contact" <p> innerHTML: "Email me at: " <hr> innerHTML: "Example"	
2	Teks HTML	<pre><html> <body> <div id="container"> <h1 class="title">Judul</h1> <p class="text">Paragraf 1</p> <p>Paragraf 2</p> Catatan A Catatan B <section> <div class="inner"> <p class="text">Paragraf Nested</p> </div> </section> </div> </body> </html></pre>
	DOM Tree:	

DOM Tree Result Insert CSS Selector Traversal Type BFS Result Count 5 Search Kedalaman Maksimal: 7 <html> <body> <div> id: "container" <h1> class: "title" innerText: "Judul" <p> class: "text" innerText: "Paragraf 1" <p> innerText: "Paragraf 2" class: "note" innerText: "Catatan A" innerText: "Catatan B" <section> <div> class: "inner" <p> class: "text" innerText: "Paragraf Nested"		
DOM Tree Result Insert CSS Selector Traversal Type BFS Result Count 5 Search Kedalaman Maksimal: 7 <html> <body> <div> id: "container" <h1> class: "title" innerText: "Judul" <p> class: "text" innerText: "Paragraf 1" <p> innerText: "Paragraf 2" class: "note" innerText: "Catatan A" innerText: "Catatan B" <section> <div> class: "inner" <p> class: "text" innerText: "Paragraf Nested"		
5	Link URL	https://quotes.toscrape.com/
	DOM Tree:	DOM Tree Result Insert CSS Selector Traversal Type BFS Result Count 5 Search Kedalaman Maksimal: 7 <html> <body> <div> id: "container" <h1> class: "title" innerText: "Judul" <p> class: "text" innerText: "Paragraf 1" <p> innerText: "Paragraf 2" class: "note" innerText: "Catatan A" innerText: "Catatan B" <section> <div> class: "inner" <p> class: "text" innerText: "Paragraf Nested"

6	File HTML (Click Test.html)	<pre> <!DOCTYPE HTML> <html lang="id"> <head> <meta charset="utf-8"> <meta http-equiv="X-UA-Compatible" content="IE=edge"> <meta name="viewport" content="width=device-width, initial-scale=1"> <title>Click Test</title> <link rel="icon" href="Assets/Sun.png"> <link href="Bootstrap/bootstrap.css" rel="stylesheet"> <link rel="stylesheet" href="style.css"> <style> .center { display: flex; justify-content: center; align-items: center; border: none; margin: 10px; } button { width: 100%; height: 300px; font-size: 3rem; border-width: 10px; border-color: #86c7ff; background-color: #c4f0ff; } p { font-size: xx-large; font-weight: bolder; text-align: center; margin: 0px; } </style> </head> <body> <nav class="navbar navbar-expand-lg yellow-bg pb-0"> <div class="container-fluid"> <div class = "pb-2"> </pre>
---	-----------------------------------	--

		<pre> <svg width="160px" viewBox="0 0 1440 420"> <path d="M 240 0 C 160 0 80 0 40 80 C 0 160 40 160 80 200 C 160 200 40 320 160 360 C 240 400 280 320 440 400 C 680 440 840 440 920 400 C 1000 320 1200 400 1280 320 C 1400 280 1320 240 1400 160 C 1440 80 1360 0 1240 40 C 1080 80 1040 80 880 40 C 760 0 720 0 560 40 C 440 80 400 40 240 0 Z" fill="white"></path> <circle cx="450" cy="220" r="170" class="yellow" fill-opacity="0.3"></circle> <circle cx="1000" cy="270" r="140" class="green" fill-opacity="0.25"></circle> <path class="blue" fill-opacity="0.25"d="M 500 220 L 800 350 L 850 70"></path> <text x="240" y="350" class="red very-big" rotate="-8">N</text> <text x="460" y="370" class="green very-big" rotate="4">A</text> <text x="650" y="360" class="pink very-big" rotate="5">F</text> <text x="900" y="380" class="yellow very-big" rotate="-3">I</text> <text x="1070" y="350" class="blue very-big" rotate="5">S</text> </svg> </div> <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation"> </button> </pre>
--	--	--

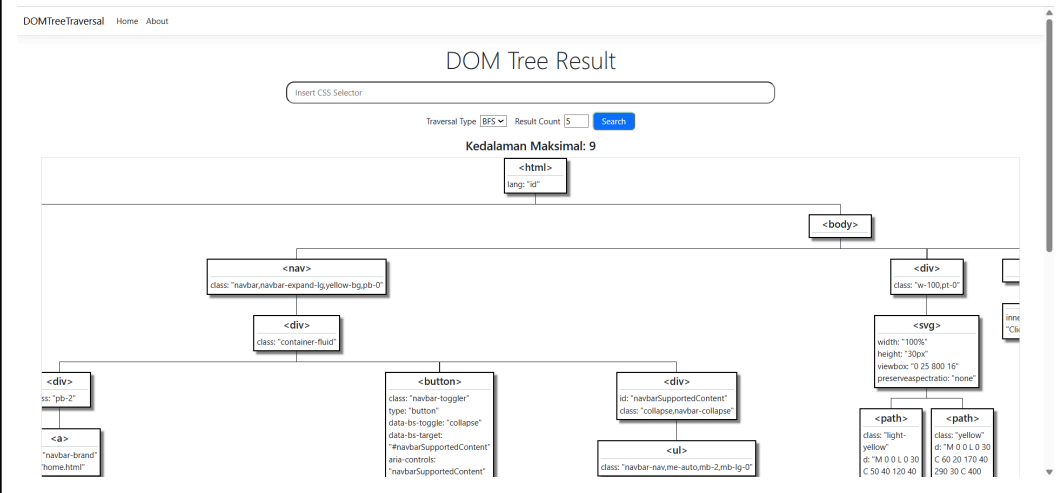
		<pre> <div >="" ><="" >about<="" >coding<="" >games<="" >home<="" >maths<="" <="" <a="" <div="" <li="" <p>click="" <path="" <svg="" <ul="" <="" a>="" aria-current="page" class="container" d="M 0 0 L 0 30 C 60 20 170 40 290 30 C 400 30 480 20 620 30 C 650 30 730 40 800 30 L 800 0 Z" div>="" height="30px" href="about.html" id="navbarSupportedContent" li>="" nav>="" p>="" path>="" pre="" preserveaspectratio="none" svg>="" test<="" ul>="" viewbox="0 25 800 16" width="100%"> </div></pre>
--	--	--

		<pre> <div class="row"> <button id="start" onclick="start()">Click here to start</button> <button id="click" style="display:none;">Click!!</button> </div> <div class="row"> <p>Time Passed : 0.000 s</p> <p>Click count : 0</p> <form class="center"> <label>Seconds</label> <input id ="seconds" type="number" style="margin-left:20px;" value="5"> </form> </div> </div> <script src="Bootstrap/bootstrap.bundle.js"></script > <script> var click = 0; var started = false; function start() { if (started) return; stared = true; var sec = document.getElementById("seconds").value; var countDownDate = new Date().getTime() + (sec * 1000); addClick(); hide(); var x = setInterval(function() { var now = new Date().getTime(); var distance = countDownDate - now; var seconds = Math.floor((distance % (1000 * 60)) / 1000); var milliseconds = (distance % 1000); </pre>
--	--	---

		<pre> if (milliseconds < 10) { document.getElementById("time").innerText = seconds + ".00" + milliseconds + " s"; } else if (milliseconds < 100) { document.getElementById("time").innerText = seconds + ".0" + milliseconds + " s"; } else { document.getElementById("time").innerText = seconds + "." + milliseconds + " s"; } if (distance < 0) { clearInterval(x); document.getElementById("time").innerText = "0.000 s"; alert (click / sec + "CPS!!!!!!"); hide(); restart(); return click; } }, 1); window.addEventListener('mousedown', addClick) } function addClick() { click++; document.getElementById("count").innerText = click; return click; } function restart() { window.removeEventListener('mousedown', addClick) started = false; click = 0; return click; </pre>
--	--	--

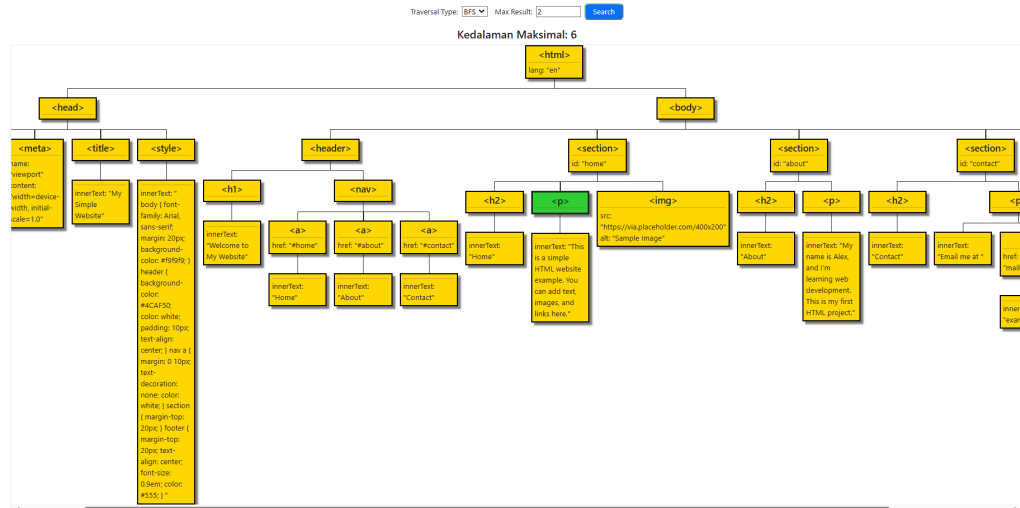
		<pre> } function hide() { var x = document.getElementById("start"); if (x.style.display === "none") { x.style.display = "block"; } else { x.style.display = "none"; } var y = document.getElementById("click"); if (y.style.display === "none") { y.style.display = "block"; } else { y.style.display = "none"; } } </script> </body> </html></pre>
--	--	--

DOM Tree:



Berikut adalah hasil kasus uji untuk CSS selector menggunakan HTML kasus uji di atas:

No.	Css Selector
1.	#home > p:



header a

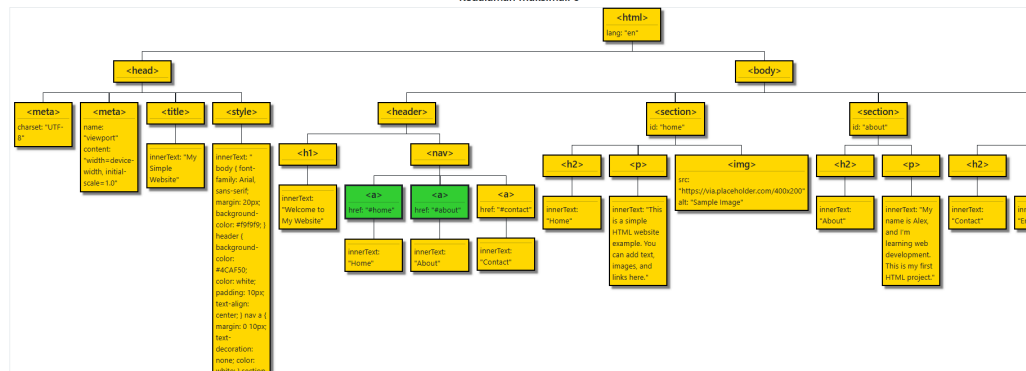
DOMTreeTraversal Home About

CSS Selector Result

header a

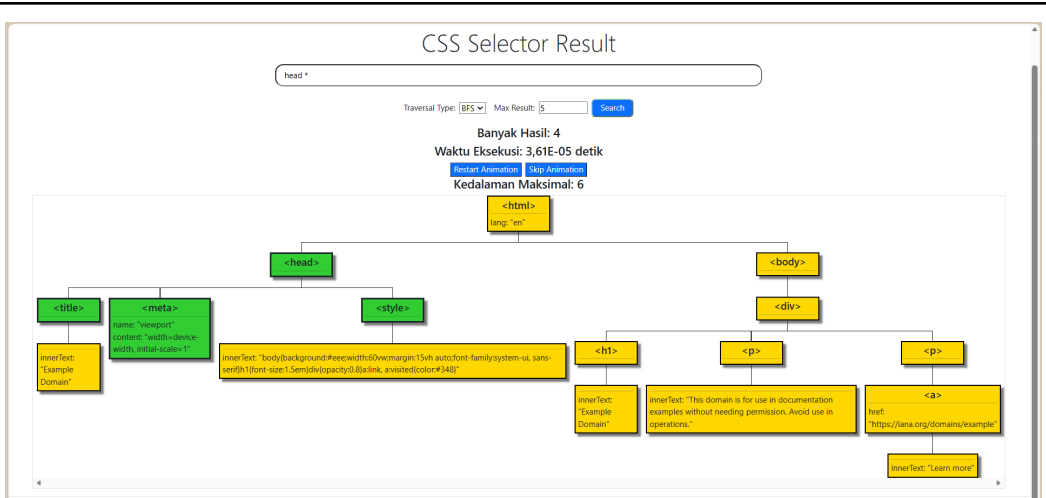
Traversal Type: **BFS** Max Result: 2

Kedalaman Maksimal: 6

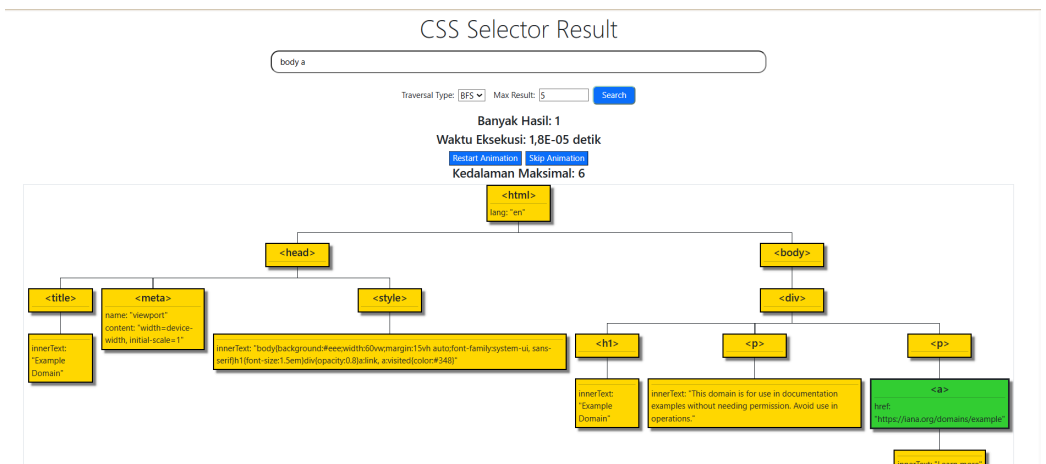


section + section

	div p Traversal Type: BFS Max Result: 2 Search Banyak Hasil: 2 Waktu Eksekusi: 0,000289 detik Restart Animation Stop Animation Kedalaman Maksimal: 7 <html> <body> <div> id: "container" <h1> class: "title" innerText: "Judul" <p> class: "text" innerText: "Paragraf 1" <p> innerText: "Paragraf 2" class: "note" innerText: "Catatan A" innerText: "Catatan B" <section> <div> class: "inner" <p> class: "text" innerText: "Paragraf Nested"
	p + span DOMTreeTraversal Home About CSS Selector Result p + span Traversal Type: BFS Max Result: 2 Search Banyak Hasil: 1 Waktu Eksekusi: 2,23E-05 detik Restart Animation Stop Animation Kedalaman Maksimal: 7 <html> <body> <div> id: "container" <h1> class: "title" innerText: "Judul" <p> class: "text" innerText: "Paragraf 1" <p> innerText: "Paragraf 2" class: "note" innerText: "Catatan A" innerText: "Catatan B" <section> <div> class: "inner" <p> class: "text"
3.	head *



body a



div > p

CSS Selector Result

div > p

Traversal Type: **BFS** Max Result: **5** Search

Banyak Hasil: 2
Waktu Eksekusi: 1,31E-05 detik
[Restart Animation](#) [Skip Animation](#)
Kedalaman Maksimal: 6

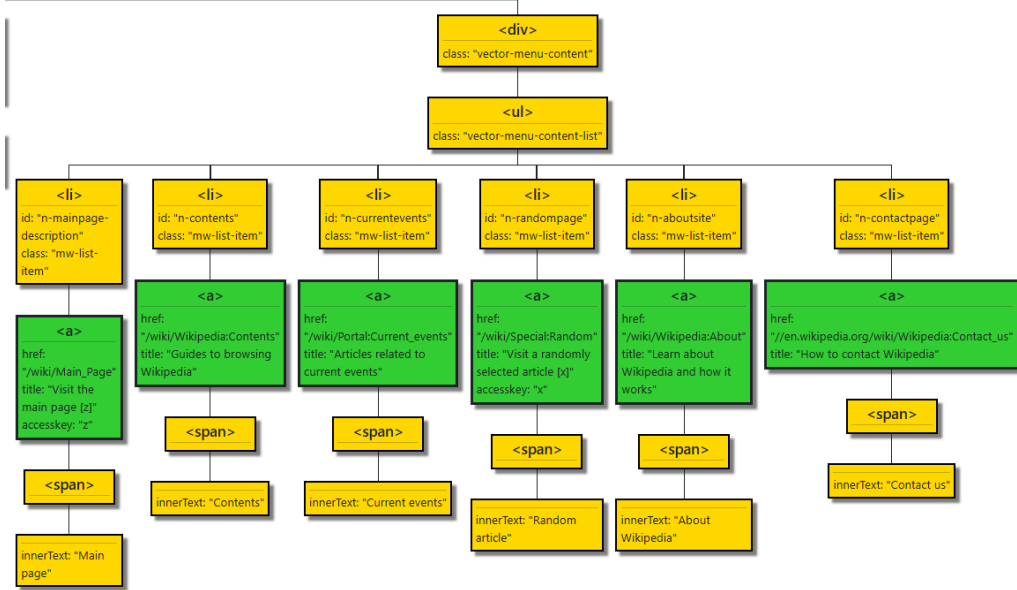
4. nav div.vector-menu-content a

CSS Selector Result

nav div.vector-menu-content a

Traversal Type: **BFS** Max Result: **100** Search

Banyak Hasil: 34
Waktu Eksekusi: 0,0004043 detik
[Restart Animation](#) [Skip Animation](#)
Kedalaman Maksimal: 21



meta + link

CSS Selector Result

meta + link

Traversal Type: Max Result:

Banyak Hasil: 2

Waktu Eksekusi: 0,0003269 detik

Kedalaman Maksimal: 21

link + meta ~ link + link

	CSS Selector Result link + meta ~ link + link Traversal Type: BFS Max Result: 10 Search Banyak Hasil: 10 Waktu Eksekusi: 0,0001234 detik Restart Animation Skip Animation Kedalaman Maksimal: 21
5.	div (bfs) DOMTreeTraversal Home About CSS Selector Result div Traversal Type: BFS Max Result: 3 Search Banyak Hasil: 3 Waktu Eksekusi: 3,55E-05 detik Restart Animation Skip Animation Kedalaman Maksimal: 11
	div (dfs)

CSS Selector Result

div

Traversal Type: BFS Max Result: 3 Search

Banyak Hasil: 3
Waktu Eksekusi: 4,53E-05 detik
Restart Animation Skip Animation
Kedalaman Maksimal: 11

```

graph TD
    html[<html>] --> head[<head>]
    html --> body[<body>]
    head --> meta[<meta>]
    head --> title[<title>]
    head --> link1[<link>]
    head --> link2[<link>]
    body --> div1[<div>]
    div1 --> div2[<div>]
    div1 --> div3[<div>]
    
```

[illegible]

6.	head *
----	--------

CSS Selector Result

head *

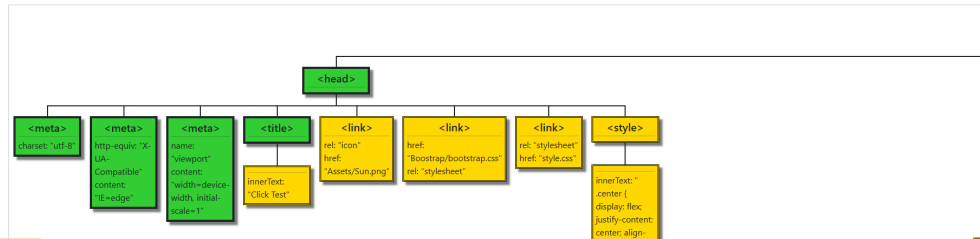
Traversal Type: Max Result:

Banyak Hasil: 5

Waktu Eksekusi: 0,0004168 detik

Restart Animation Skip Animation

Kedalaman Maksimal: 9



- ul > li a *

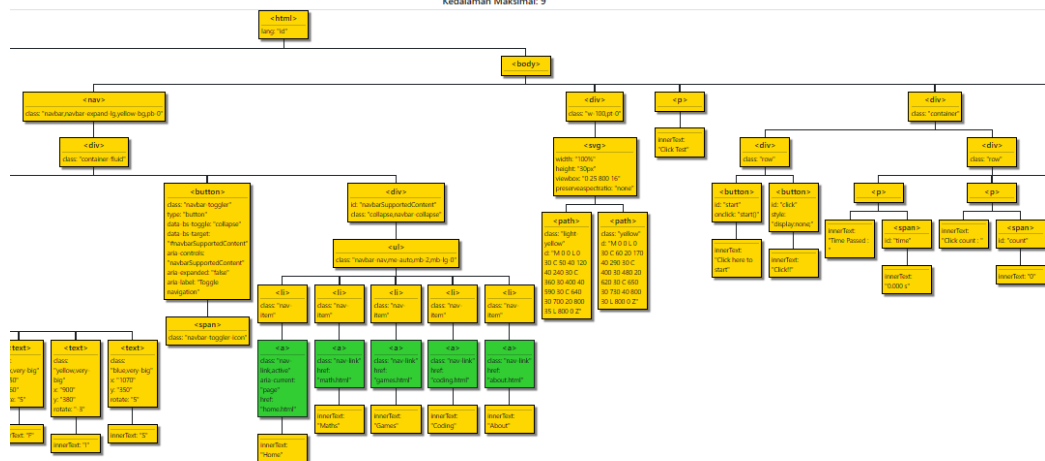
CSS Selector Result

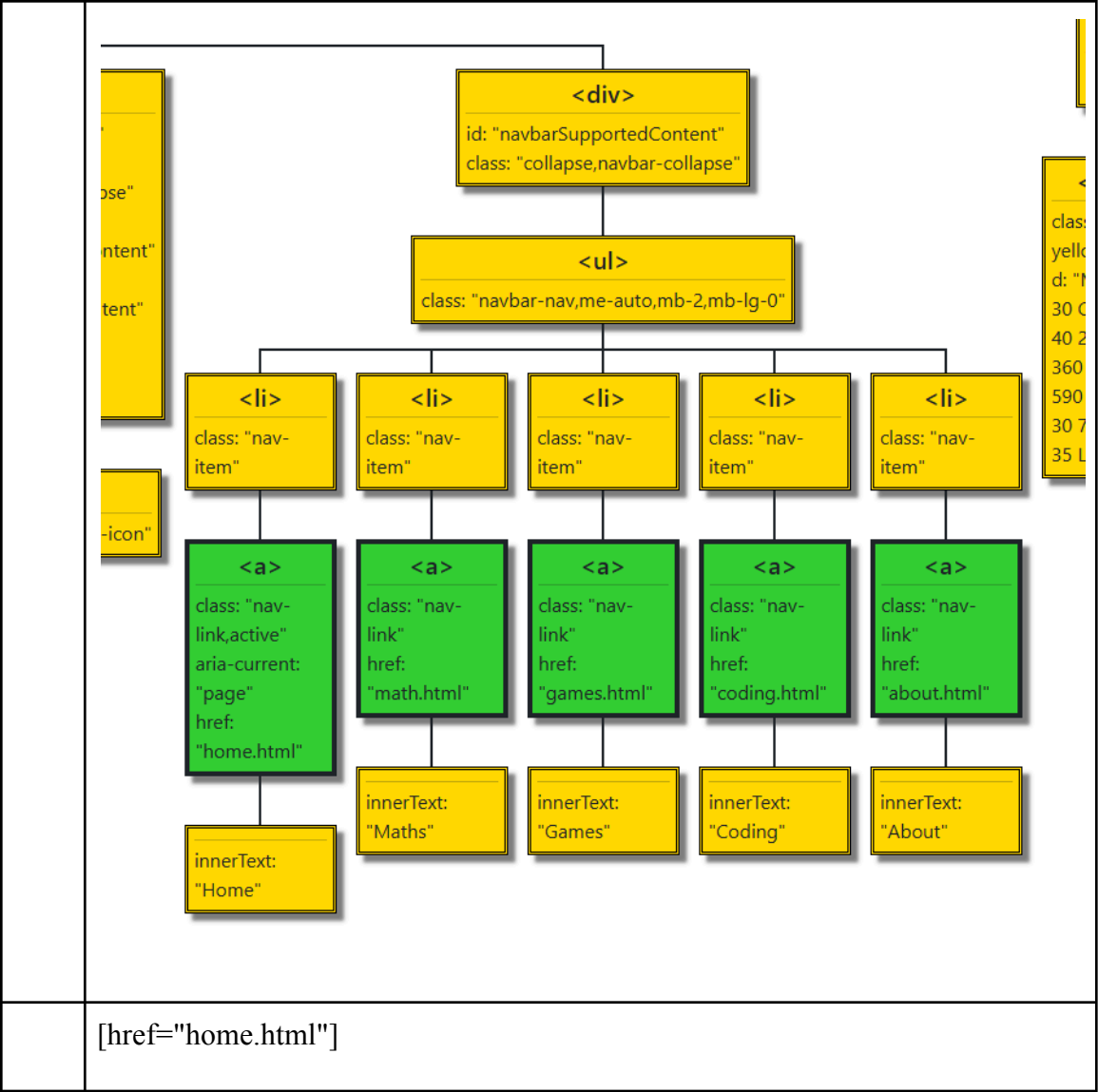
Traversal Type: Max Result:

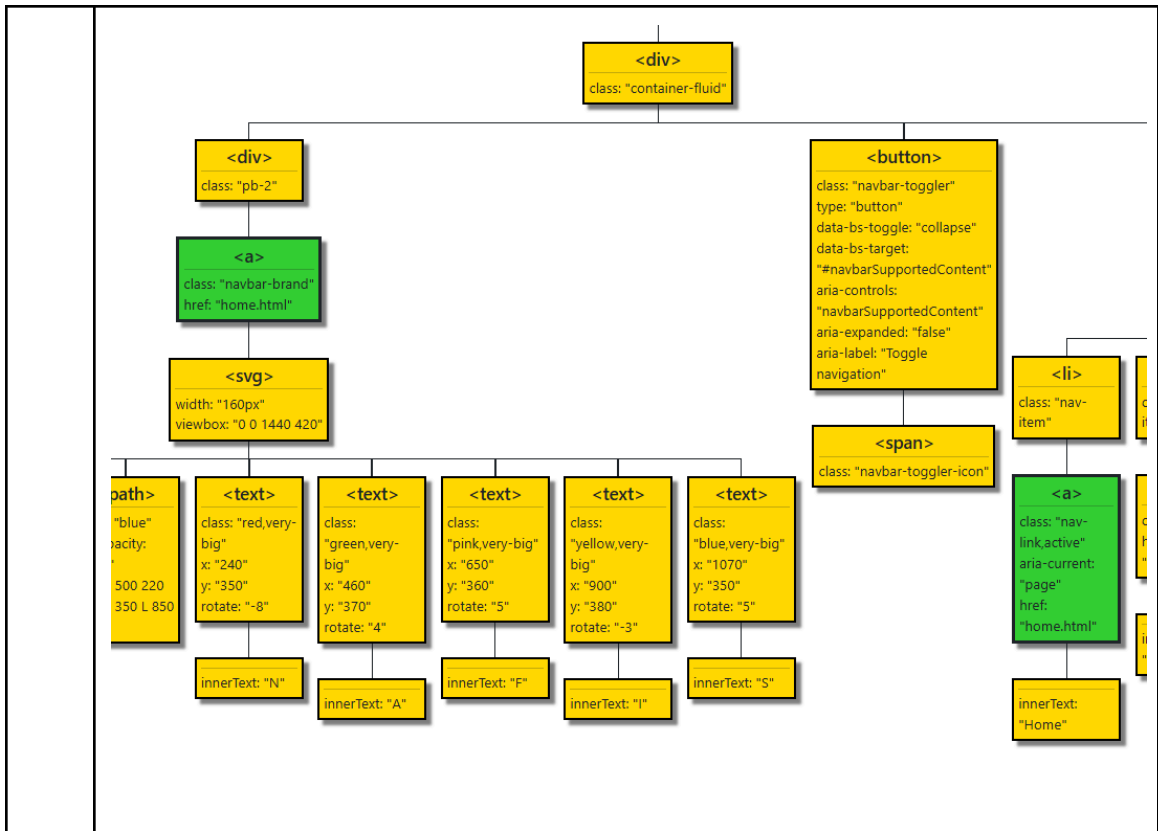
Banyak Hasil: 5

Waktu Eksekusi: 0,0033284 detik

Restart Animation Skip Animation







BAB 5

Kesimpulan dan Saran

1. Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa aplikasi traversal pohon HTML berbasis Document Object Model (DOM) berhasil dibangun sesuai dengan ekspektasi. Aplikasi mampu menerima input dokumen HTML dari berbagai sumber, yaitu URL website, file HTML lokal, maupun teks HTML manual, kemudian memproses dokumen tersebut menjadi struktur data pohon DOM yang dapat ditelusuri secara terprogram.

Implementasi HTML Parser yang dikembangkan telah berhasil melakukan proses tokenisasi dan parsing dokumen HTML menjadi node-node yang saling terhubung melalui relasi parent-child. Hasil parsing tersebut kemudian diserialisasikan ke dalam format JSON sehingga dapat digunakan baik oleh backend maupun frontend untuk kebutuhan visualisasi struktur DOM.

Selain itu, modul CSS Selector Parser dan Matcher juga telah berhasil diimplementasikan untuk mengenali query selector yang dimasukkan pengguna serta menentukan node-node DOM yang memenuhi kriteria pencarian. Dengan adanya modul ini, proses pencarian elemen HTML dapat dilakukan secara spesifik berdasarkan tag, id, class, maupun kombinasi selector.

Pada inti aplikasi, algoritma Breadth First Search (BFS) dan Depth First Search (DFS) berhasil diterapkan untuk melakukan traversal terhadap seluruh node pada DOM Tree. Kedua algoritma tersebut tidak hanya mampu mengunjungi node-node DOM sesuai pola traversal masing-masing, tetapi juga dapat digunakan untuk mencari node yang sesuai dengan query CSS Selector. BFS melakukan penelusuran secara melebar berdasarkan level, sedangkan DFS melakukan penelusuran secara mendalam pada setiap cabang. Perbedaan pola traversal ini berhasil divisualisasikan melalui traversal log dan highlight jalur penelusuran pada frontend.

Dengan demikian, tujuan utama tugas besar ini, yaitu membangun aplikasi web untuk traversal dan pencarian elemen pada pohon DOM menggunakan algoritma BFS dan DFS, telah berhasil dicapai.

2. Saran

Terdapat beberapa aspek yang dapat dikembangkan lebih lanjut agar sistem menjadi lebih optimal dan lebih mendekati implementasi browser maupun selector engine yang sebenarnya.

Pertama, dukungan terhadap CSS Selector masih dapat diperluas dengan menambahkan selector yang lebih kompleks, seperti pseudo-class selector, attribute selector lanjutan, serta combinator yang lebih lengkap. Dengan penambahan ini, aplikasi akan mampu menangani query pencarian yang lebih variatif.

Kedua, performa aplikasi dapat dioptimalkan untuk menangani dokumen HTML berukuran besar. Pada DOM yang sangat kompleks, traversal BFS dan DFS akan mengunjungi node dalam jumlah besar sehingga waktu proses dapat meningkat dan tampilan akan rumit. Oleh karena itu, dapat dipertimbangkan penerapan optimasi pruning atau indexing node tertentu.

Lampiran

Link repositori Github: https://github.com/NafisKreatif/Tubes2_OneStack

Pernyataan Penggunaan AI

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Muhammad Nafis Habibi



Muhammad Jordan Ferimeison



Wildan Abdirrahman Ghazali

Tabel Poin-Poin Pengerjaan:

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	V	
2	Aplikasi berhasil dijalankan	V	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	V	
4	Aplikasi dapat melakukan scraping terhadap web pada input	V	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	V	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	V	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	V	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	V	
9	[Bonus] Membuat video		X
10	[Bonus] Deploy aplikasi		X
11	[Bonus] Implementasi animasi pada penelusuran pohon	V	
12	[Bonus] Implementasi multithreading		X
13	[Bonus] Implementasi LCA Binary Lifting		X

Tabel Pembagian Tugas

No	Nama	NIM	Tugas
1	Muhammad Nafis Habibi	13524018	Front-end
2	Muhammad Jordan Ferimeison	13524047	CSS Selector
3	Wildan Abdurrahman Ghazali	13524054	HTML Parser